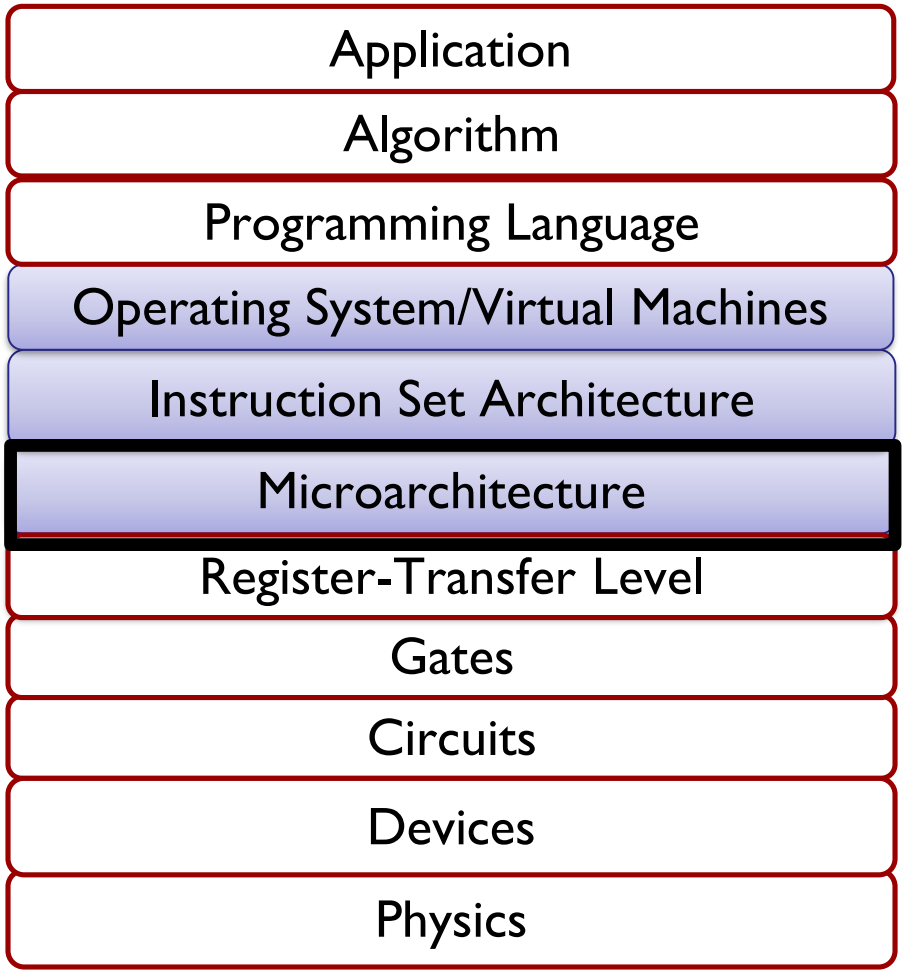
THE UNIVERSITY OF CHICAGO

The Memory Hierarchy

CS144

Prof. Yanjing Li



Current topics:

- Intro to memory hierarchy
- Caches

Instruction Type	Frequency
Memory → Register Move	22%
Conditional Branch	20%
Compare	16%
Register → Memory Move	12%
Add	8%
And	6%
Sub	5%
Register → Register Move	4%
Function call	1%
Function return	1%
Total	**96%**

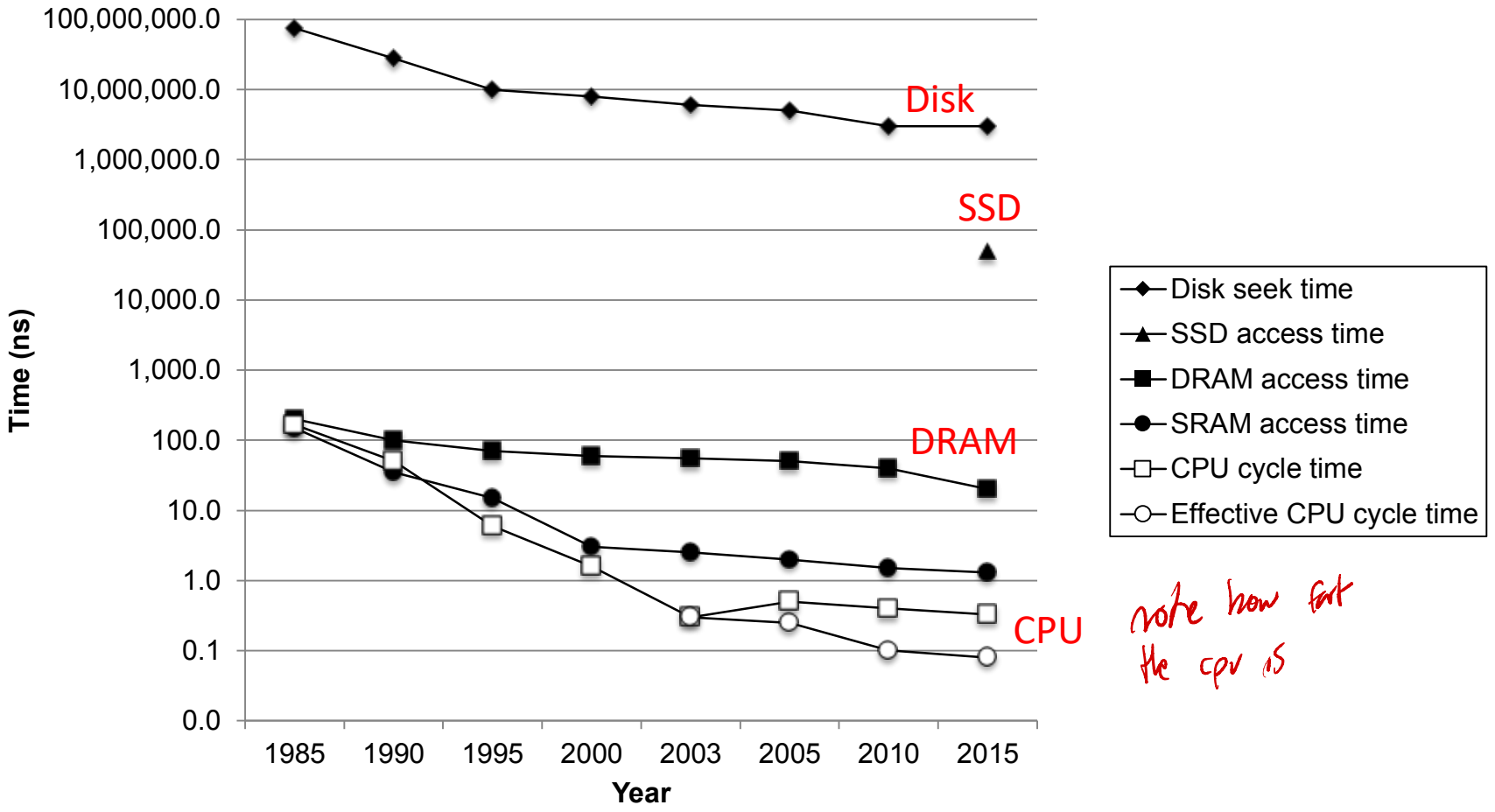
From H&P: Computer Architecture a Quantitative Approach

> 34% of instructions access data memory!

100% of instructions are stored in instruction memory!

The CPU-Memory Gap

The gap widens between DRAM, disk, and CPU speeds.



Synthesize This

> 34% of instructions access data memory

But if our memory is so slow, this is no good that we use it, right?

100% of instructions access instruction memory

Memory is many orders of magnitude slower than the processor

For example: Four-issue 2 GHz superscalar accessing 100 ns DRAM could execute 800 instructions during the time for one memory access!

These numbers don't make sense

There must be a way to speed up memory access!

Are All Memories Slow?

No.

But, bigger means slower ⍰ Fundamental Tradeoff

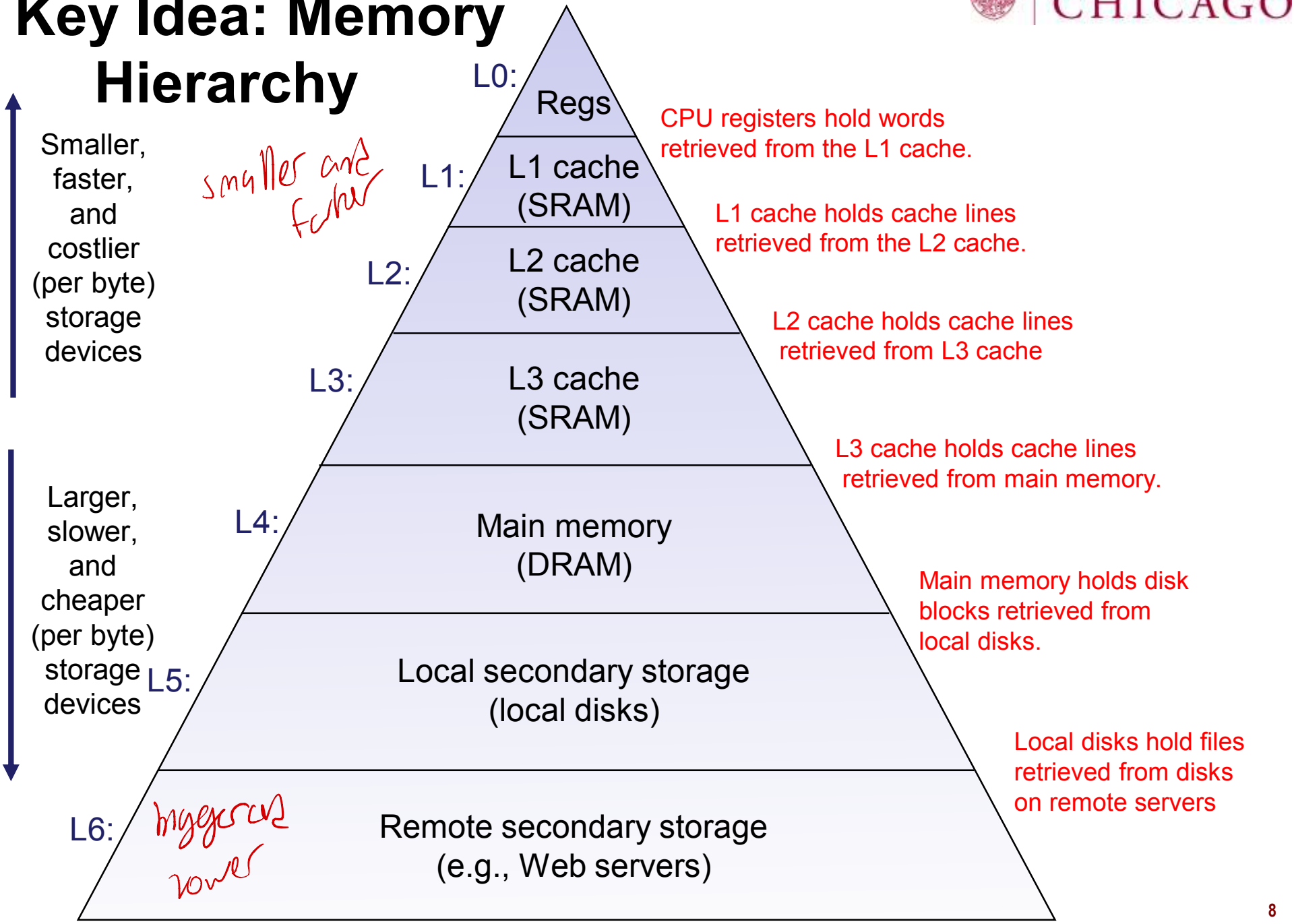
Technology differences

Hard disk (slow) vs. SSD vs. DRAM vs. SRAM (fast)

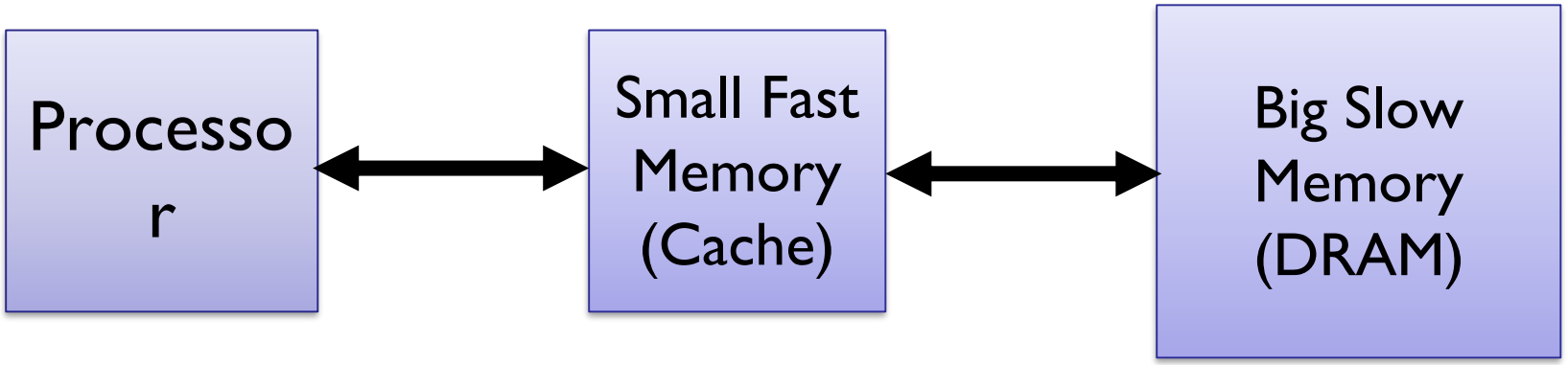
Just physics

Signals in longer wires take longer to travel

Key Idea: Memory Hierarchy



Memory Hierarchy



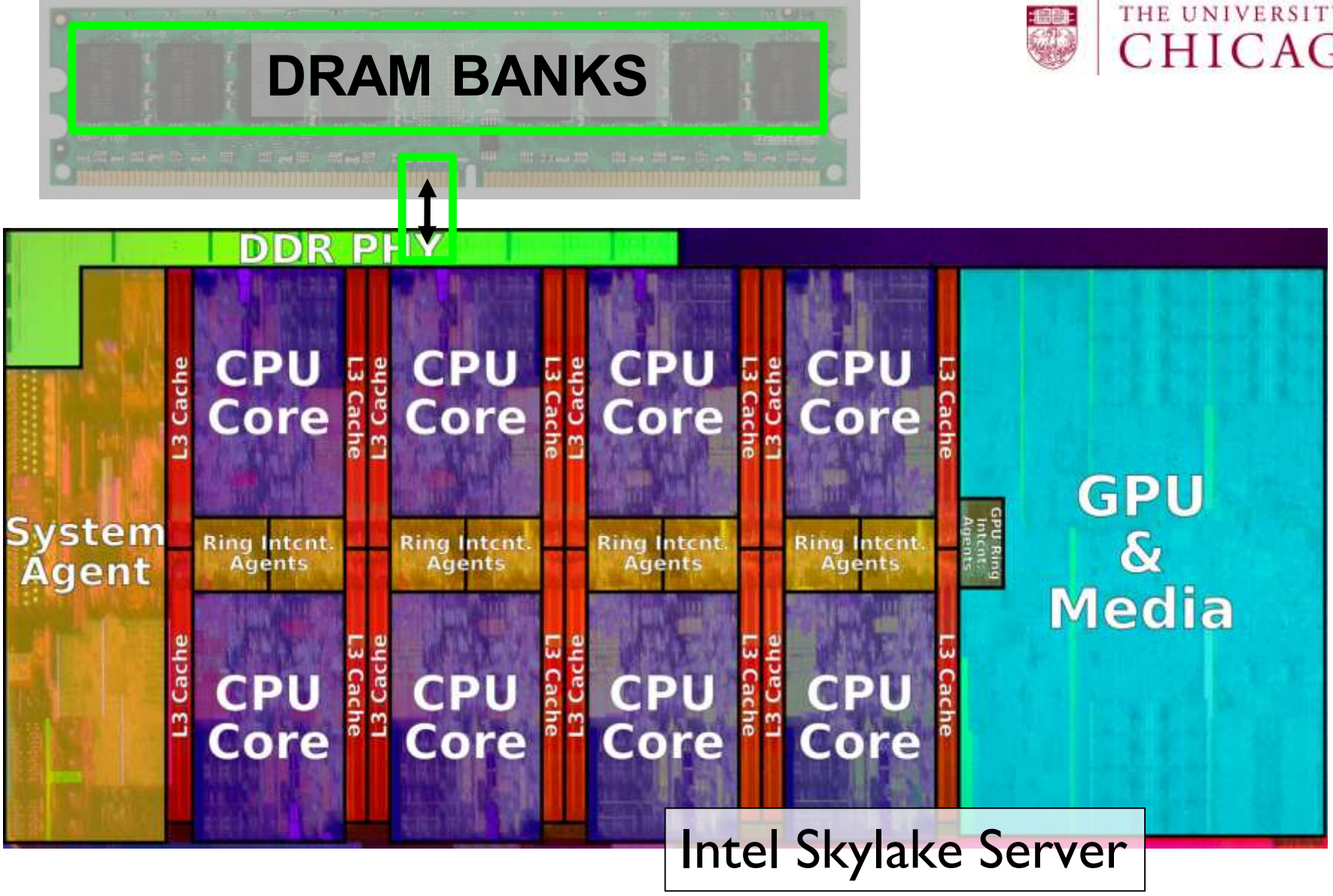
Capacity: Register << Cache << DRAM

Speed: Register >> Cache >> DRAM

On a data access:

if data is in fast memory -> low-latency access (to cache or register)

if data is not in fast memory -> long-latency access to DRAM



- **CPU Core (incl L1, L2), L3, and DRAM**

Why Does Memory Hierarchy Work?

Memory hierarchies only work if …

The small, fast memory actually stores data that is used/re-used by the processor at a given time

Do they???

Locality!

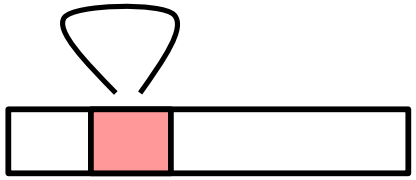
The key to bridging this CPU-Memory gap is a fundamental property of computer programs known as locality

Locality

Principle of Locality: Programs tend to use data and instructions with addresses near or equal to those they have used recently

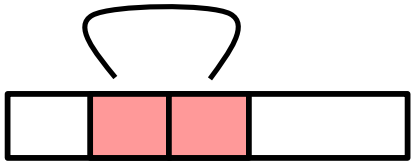
Temporal locality:

Recently referenced items are likely
to be referenced again in the near future



Spatial locality:

Items with nearby addresses tend
to be referenced close together in time



Locality Example

```
sum = 0;
for (i = 0; i < n; i++)
        sum += a[i];
return sum;
```

Data references

Reference array elements in succession (stride-1 reference pattern).

Reference variable `sum` each iteration.

Instruction references

Reference instructions in sequence.

Cycle through loop repeatedly.

Spatial locality

Temporal locality

Spatial locality

Temporal locality

Locality Example

Claim: Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer.

Question: Does the function exhibit good locality?

No, we want to access the array elements in order

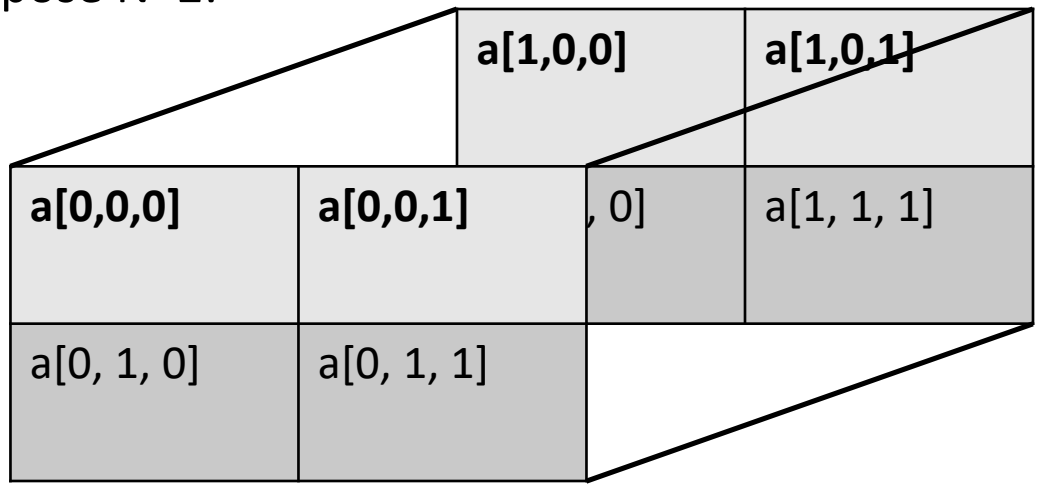
```
int sum_array_3d(int a[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += a[k][i][j];
    return sum;
}
```

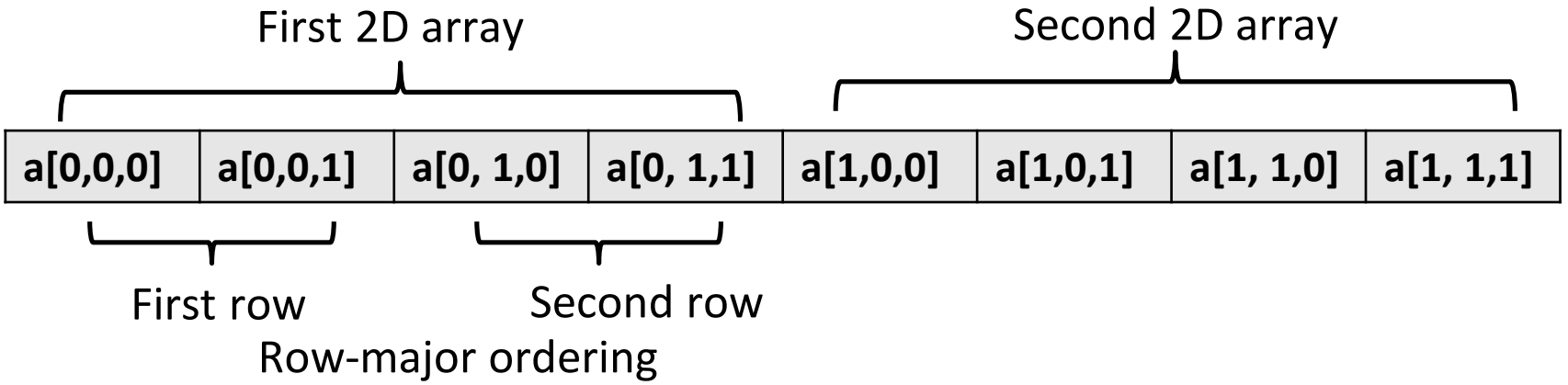
Locality Example

A 3D array is an array of 2D arrays.

Suppose N=2:



Memory layout: contiguous locations



Locality Example

```
int sum_array_3d(int a[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += a[k][i][j];

    return sum;
}
```

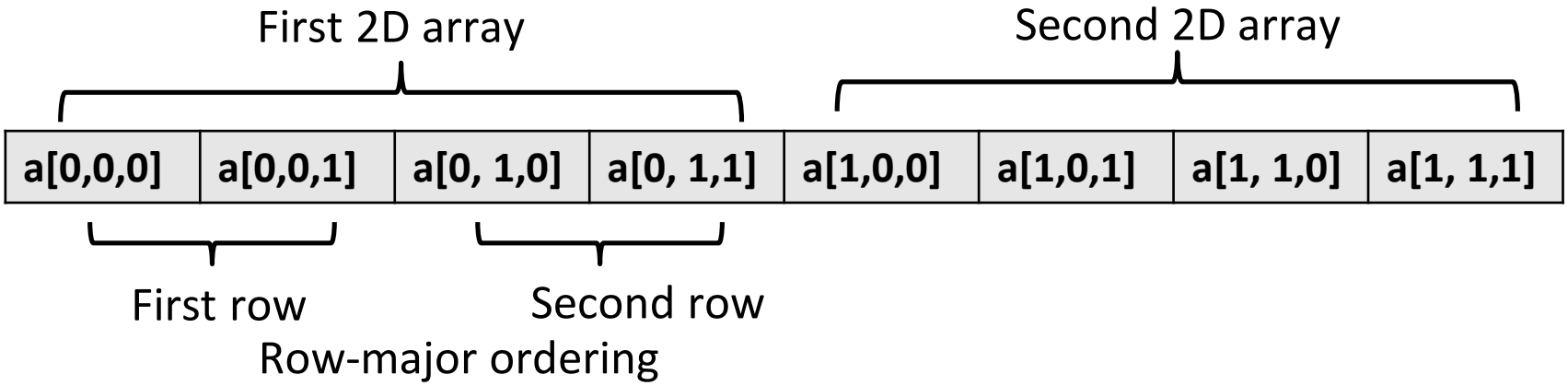
NOT good spatial locality

Access order:

a[0][0][0]
a[1][0][0]
a[0][0][1]
a[1][0][1]
a[0][1][0]
a[1][1][0]
a[0][1][1]
a[1][1][1]

not in order in memory

Memory layout: contiguous locations



Now, Back to Memory Hierarchy

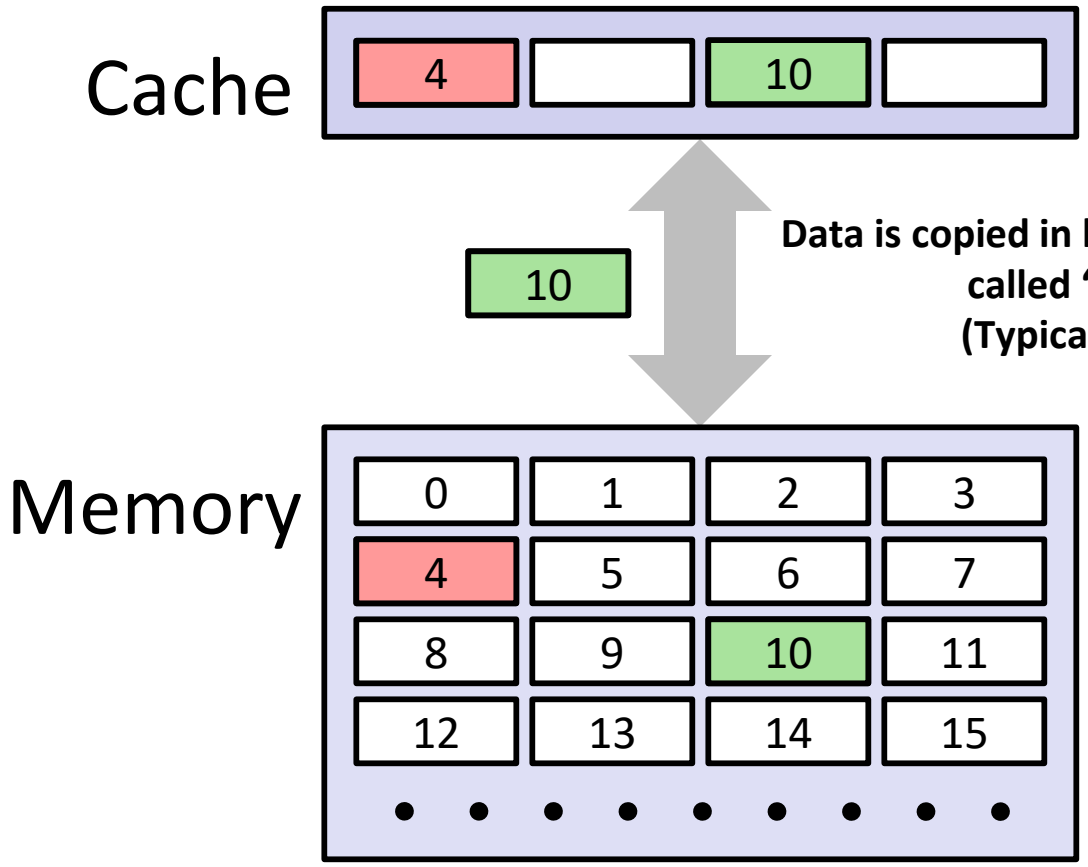
- Why do memory hierarchies work?

  Is because program have locality

- *Caches:* A smaller, faster memory that acts as a staging area for a subset of the data in a larger, slower memory (Fundamental idea of a memory hierarchy).
- multiple levels of caches exist in a system. For each k, the faster, smaller device at level k serves as a cache for the larger, slower device at level k+1.
- Designed to take advantage of program locality: programs tend to access the data at level k more often than they access the data at level k+1.
  - Thus, the storage at level k+1 can be slower, and thus larger and cheaper per bit.

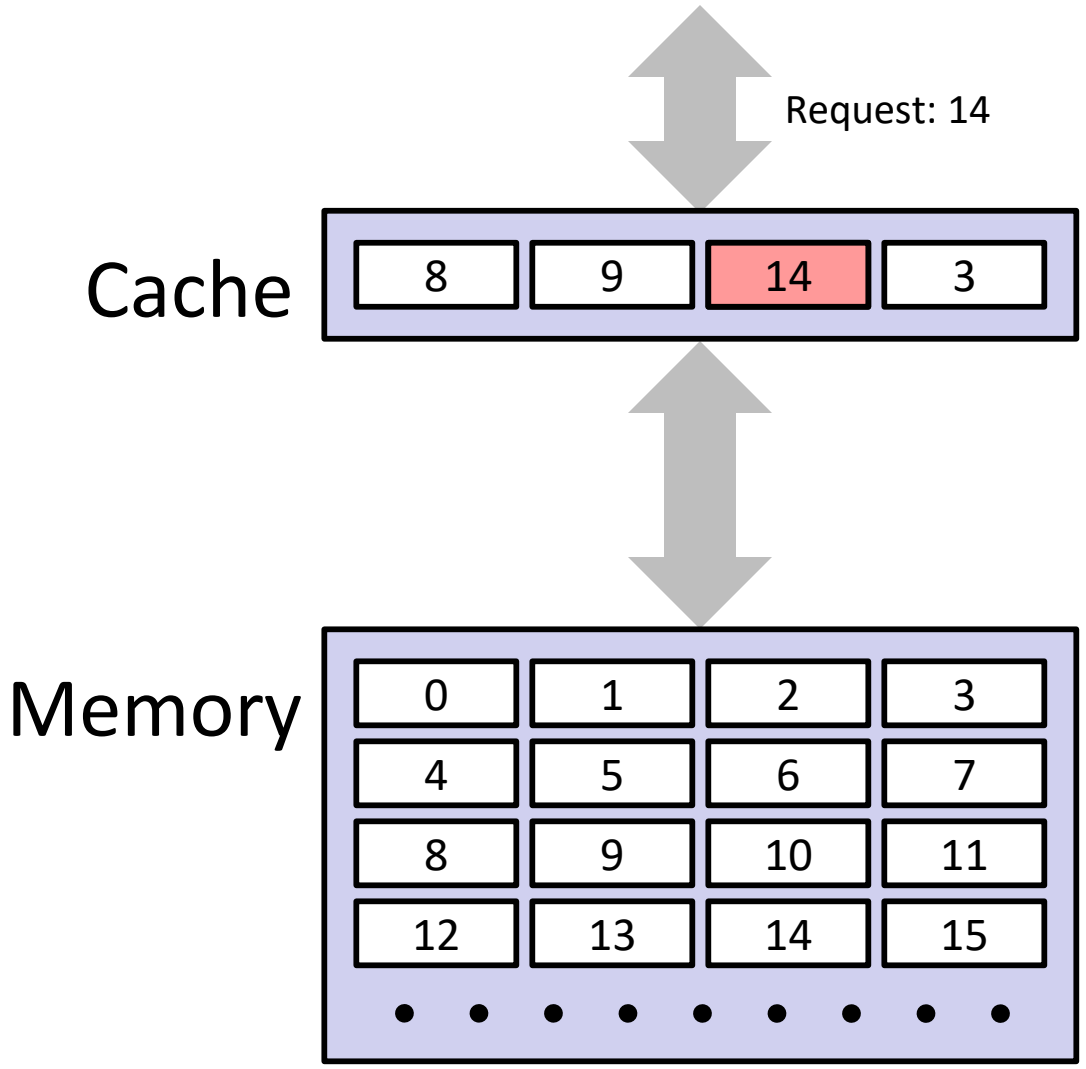
- The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

General Cache Concepts



which block is the data in? Then bring in the block of memory to the cache.

General Cache Concepts: Hit



Data in block b is needed

Block b is in cache:

Hit!

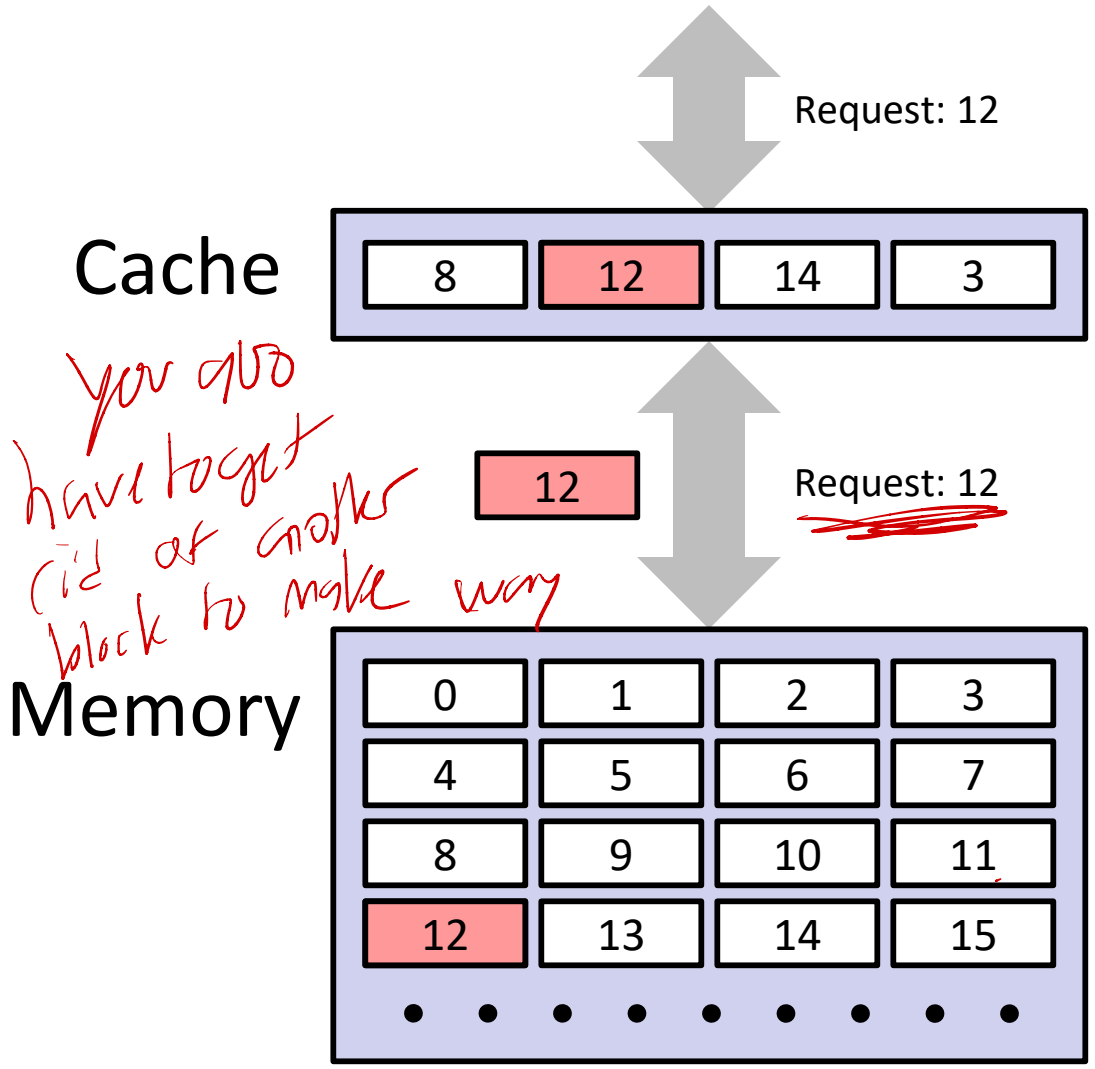
When is already in your cache

The degree to which the cache exploits locality can be quantified as *hit rate*

=

hits / # accesses

General Cache Concepts: Miss



Data in block b is needed

Block b is not in cache:
Miss!

Block b is fetched from *memory*

The degree to which the cache fails to exploit locality is its *miss rate* =
misses / # accesses

Basic Flow for a "Mem ? Reg" mov